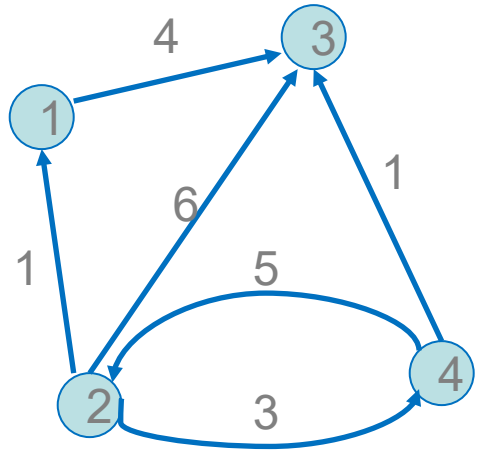
All Pairs Shortest Path

- Tabulate distances between pairs of points on the map
  - For scheduling airlines/trains, etc.
- Cast in graph terms
  - For any two vertices, tells us the shortest path between them
- Floyd's Algorithm
  - Same idea as Warshall's Algorithm

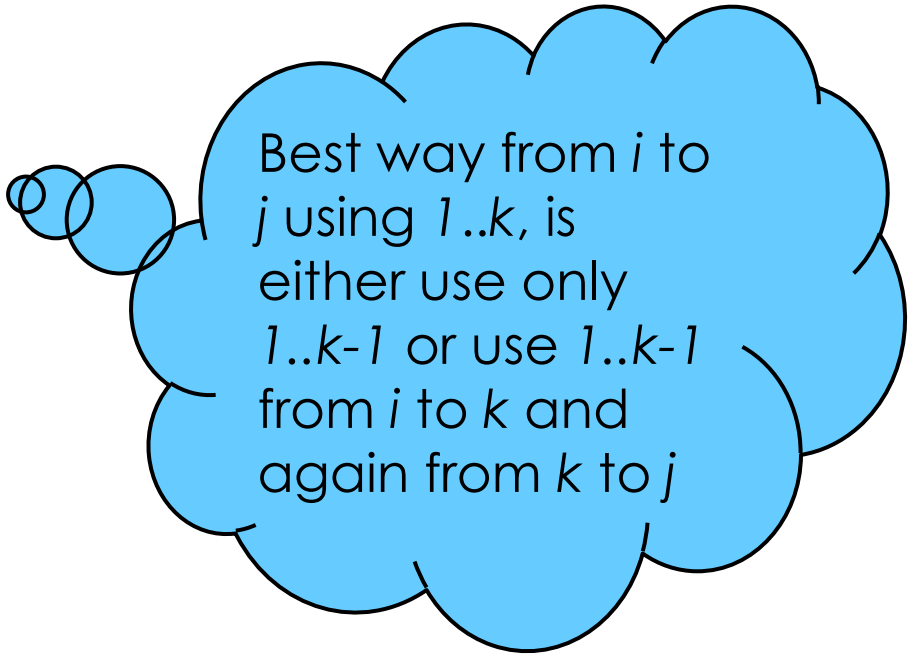
Floyd's Algorithm

- In a weighted graph (directed or undirected), find shortest paths between every pair of vertices
- Construct solution using series of matrices $D^{(0)}$, $D^{(1)}$, …, $D^{(n)}$
  - Using an initial subset of the vertices as intermediaries
  - Same idea as Warshall
- Case study graph on right

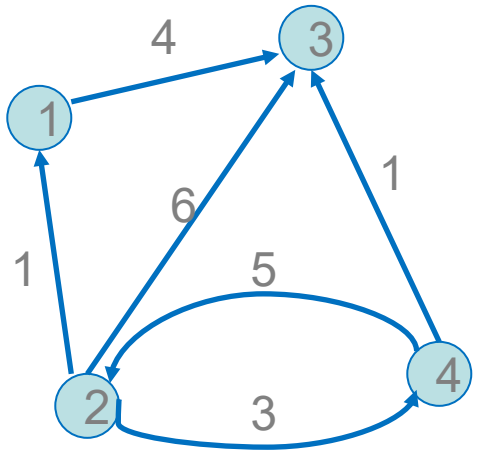


Floyd's Idea

- Initialisation:
  - $d_{ij}^{(0)} = w_{ij}$ direct
  - This is the weighted adjacency graph
  - No intermediate vertex from i to j
- Iteration:
  - $d_{ij}^{(k)} = \mathbf{min}(d_{ij}^{(k-1)} , d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ if $k \geq 1$
- Termination:
  - Stop at k = n
  - All intermediate vertices are in set {1, 2, …, n}



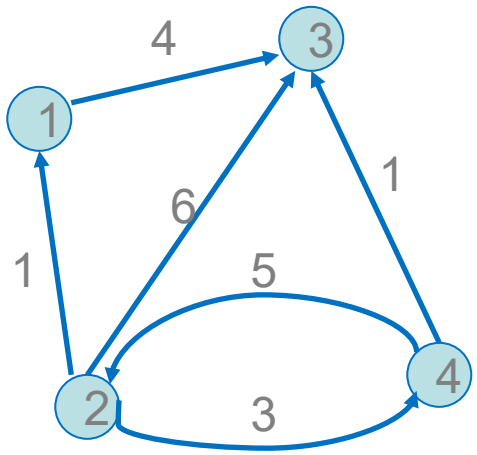
Floyd's Algorithm Example: $D^{(0)}$



$$D^0 = W = \begin{matrix} 0 & \infty & 4 & \infty \\ 1 & 0 & 6 & 3 \\ \infty & \infty & 0 & \infty \\ \infty & 5 & 1 & 0 \end{matrix}$$

- Initialise with the weighted adjacency matrix. Place infinities where there is no directed edge

Floyd's Algorithm Example: $D^{(1)}$

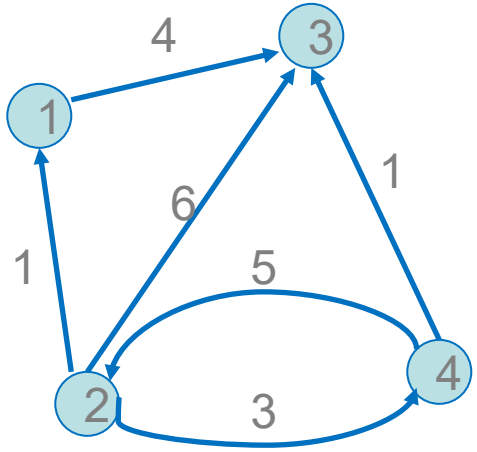


$$D^0 = W = \begin{bmatrix} 0 & \infty & 4 & \infty \\ 1 & 0 & 6 & 3 \\ \infty & \infty & 0 & \infty \\ \infty & 5 & 1 & 0 \end{bmatrix}$$

$$D^1 = \begin{bmatrix} 0 & \infty & 4 & \infty \\ 1 & 0 & 5 & 3 \\ \infty & \infty & 0 & \infty \\ \infty & 5 & 1 & 0 \end{bmatrix}$$

- At (i, j) take min of column j in row 1 + row i in column 1 and the current element

Floyd's Algorithm Example: $D^{(2)}$

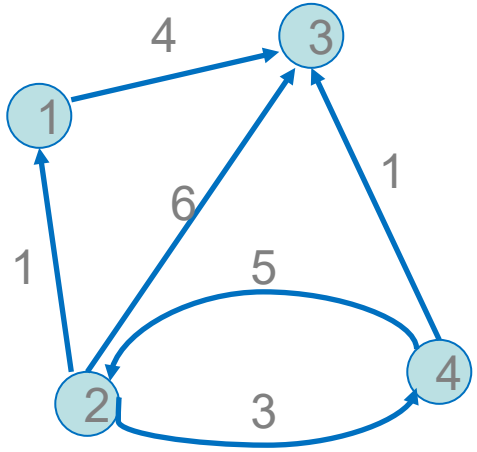


$$D^1 = \begin{matrix} 0 & \infty & 4 & \infty \\ 1 & 0 & 5 & 3 \\ \infty & \infty & 0 & \infty \\ \infty & 5 & 1 & 0 \end{matrix}$$

$$D^2 = \begin{matrix} 0 & \infty & 4 & \infty \\ 1 & 0 & 5 & 3 \\ \infty & \infty & 0 & \infty \\ 6 & 5 & 1 & 0 \end{matrix}$$

- At (i, j) take min of column j in row 2 + row i in column 2 and the current element

Floyd's Algorithm Example: $D^{(3)}$

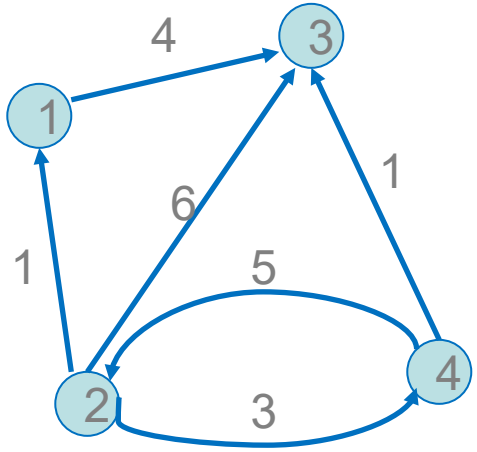


$$D^2 = \begin{matrix} 0 & \infty & 4 & \infty \\ 1 & 0 & 5 & 3 \\ \infty & \infty & 0 & \infty \\ 6 & 5 & 1 & 0 \end{matrix}$$

$$D^3 = \begin{matrix} 0 & \infty & 4 & \infty \\ 1 & 0 & 5 & 3 \\ \infty & \infty & 0 & \infty \\ 6 & 5 & 1 & 0 \end{matrix}$$

- At (i, j) take min of column j in row 3 + row i in column 3 and the current element

Floyd's Algorithm Example: $D^{(4)}$



$$D^3 = \begin{matrix} 0 & \infty & 4 & \infty \\ 1 & 0 & 5 & 3 \\ \infty & \infty & 0 & \infty \\ 6 & 5 & 1 & 0 \end{matrix}$$

$$D = D^4 = \begin{matrix} 0 & \infty & 4 & \infty \\ 1 & 0 & 4 & 3 \\ \infty & \infty & 0 & \infty \\ 6 & 5 & 1 & 0 \end{matrix}$$

- At (i, j) take min of column j in row 4 + row i in column 4 and the current element

Floyd's Algorithm Code

```
D ← W
for k ← 1 to n do
    for i  ← 1 to n do
        for j  ← 1 to n do
            D[i,j] ← min( D[i,j],  D[i,k] + D[k,j] )
return D
```

- Complexity:
  - Time efficiency in $\Theta(n^3)$
  - Space efficiency in $\Theta(n^2)$
  - Similar to Warshall

Extracting the Shortest Paths

- Dynamic Programming often provides the value of the optimium but not other information – needs to be extracted
- For Floyd's algorithm, can create a n x n matrix P (set to 0's initially) that is updated if k used as intermediate vertex: P[i,j] ← k (set when distance is reduced using k in min() step in Floyd algorithm)
  - If (D[i,k] + D[k,j] < D[i,j)
    P[i,j] = k; // intermediate node k used from i to j

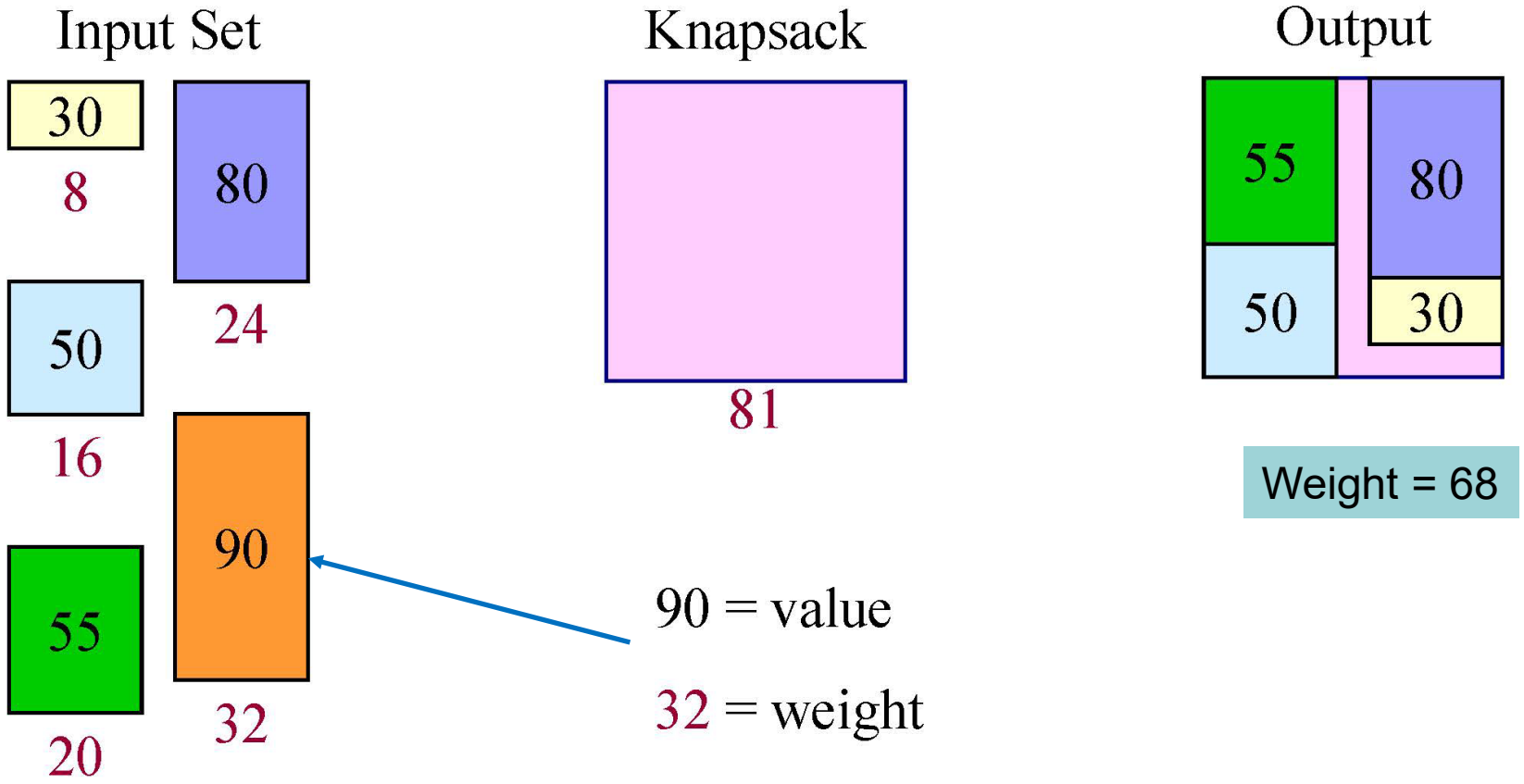
```
// input matrix P, i = start, j = end
// output shortest path nodes between i and j
Algorithm ShortestPath(i, j, P[1..n, 1..n)
 k ← P[i,j]
 if k != 0
    ShortestPath(i,k)
    print(k)
    ShortestPath(k,j)
```

Reminder: Principle

- Construct table of all subproblem results
  - initialized entries of smallest subproblems
  - remainder filled in following a precise order
    - corresponding to increasing subproblems size
    - use only entries that have already been computed
- Each entry is calculated exactly once
- Final value computed is solution to the initial problem
- Use iteration (and extra space for saved results)
  - *never* recursion

Knapsack Problem Reminder

- Fitting items with values and weights into a limited weight-capacity knapsack, such that value is maximized



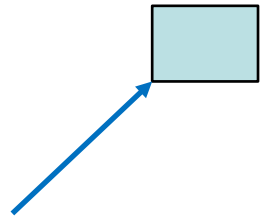
Knapsack Solution Structure

- Sub-problem definition:
  - *i* items: $1 \leq i \leq n$
  - Weights: $w_1, \ldots, w_i$
  - Values: $v_1, \ldots, v_i$
  - Knapsack size: $j - 1 \leq j \leq W$
- Data Structure:
  - Create a table *V* of sub-problem solutions
  - Where V[i,j] is the value of the optimal subset of the first *i* items that fit into a knapsack of size *j*

capacity j

Items i

	0	**1**	**2**	**3**	**4**	**5**
0	0	0	0	0	0	0
1	0					
2	0					
3	0					
4	0					



Entry V[3,4] potentially has items up to and including i=3 and capacity j=4

Knapsack Recurrence Relation

- For entry V[i,j]

  Either don't include item *i*
  - (because it doesn't fit or isn't optimal)
  - Use previous optimal packing
  - i.e., V[i,j] = V[i-1,j]

  Or do include item *i*
  - Use optimal solution for knapsack with i-1 items that is smaller by the weight of item i
  - V[i,j] = V[i-1, j-w_i] + v_i

Initialisation:

$$V[0,j] = 0$$
$$V[i,0] = 0$$

Recurrence:

$$V[i,j] = \begin{cases} V[i-1,j] & if\ j < w_i \\ \max(V[i-1,j], V[i-1,j-w_i] + v_i) & else \end{cases}$$

Knapsack Example: i=1

$$V[i,j] = \begin{cases} V[i-1,j] & if\ j < w_i \\ \max(V[i-1,j], V[i-1,j-w_i] + v_i) & else \end{cases}$$

capacity j

		0	1	2	3	4	5
	0	0	0	0	0	0	0
w_1=2, v_1=12	**1**	0	0	12	12	12	12
w_2=1, v_2=10	**2**	0					
w_3=3, v_3=20	**3**	0					
w_4=2, v_4=15	**4**	0					

Items i

initialisation

initialisation

Knapsack Example: i=2

$$V[i,j] = \begin{cases} V[i-1,j] & if\ j < w_i \\ \max(V[i-1,j], V[i-1,j-w_i] + v_i) & else \end{cases}$$

capacity j

Items i

		0	1	2	3	4	5
	0	0	0	0	0	0	0
w_1=2, v_1=12	**1**	0	0	12	12	12	12
w_2=1, v_2=10	**2**	0	10	12	22	22	22
w_3=3, v_3=20	**3**	0					
w_4=2, v_4=15	**4**	0					

max(V[1,3],V[1,2]+10)
= max(12, 22) = 22

Knapsack Example: i=3

$$V[i,j] = \begin{cases} V[i-1,j] & if\ j < w_i \\ \max(V[i-1,j], V[i-1,j-w_i] + v_i) & else \end{cases}$$

capacity j

Items i

		0	1	2	3	4	5
	0	0	0	0	0	0	0
w_1=2, v_1=12	**1**	0	0	12	12	12	12
w_2=1, v_2=10	**2**	0	10	12	22	22	22
w_3=3, v_3=20	**3**	0	10	12	22	30	32
w_4=2, v_4=15	**4**	0					

max(V[2,4],V[2,1]+20)
= max(22, 30) = 30

Knapsack Example: i=4

$$V[i,j] = \begin{cases} V[i-1,j] & if\ j < w_i \\ \max(V[i-1,j], V[i-1,j-w_i] + v_i) & else \end{cases}$$

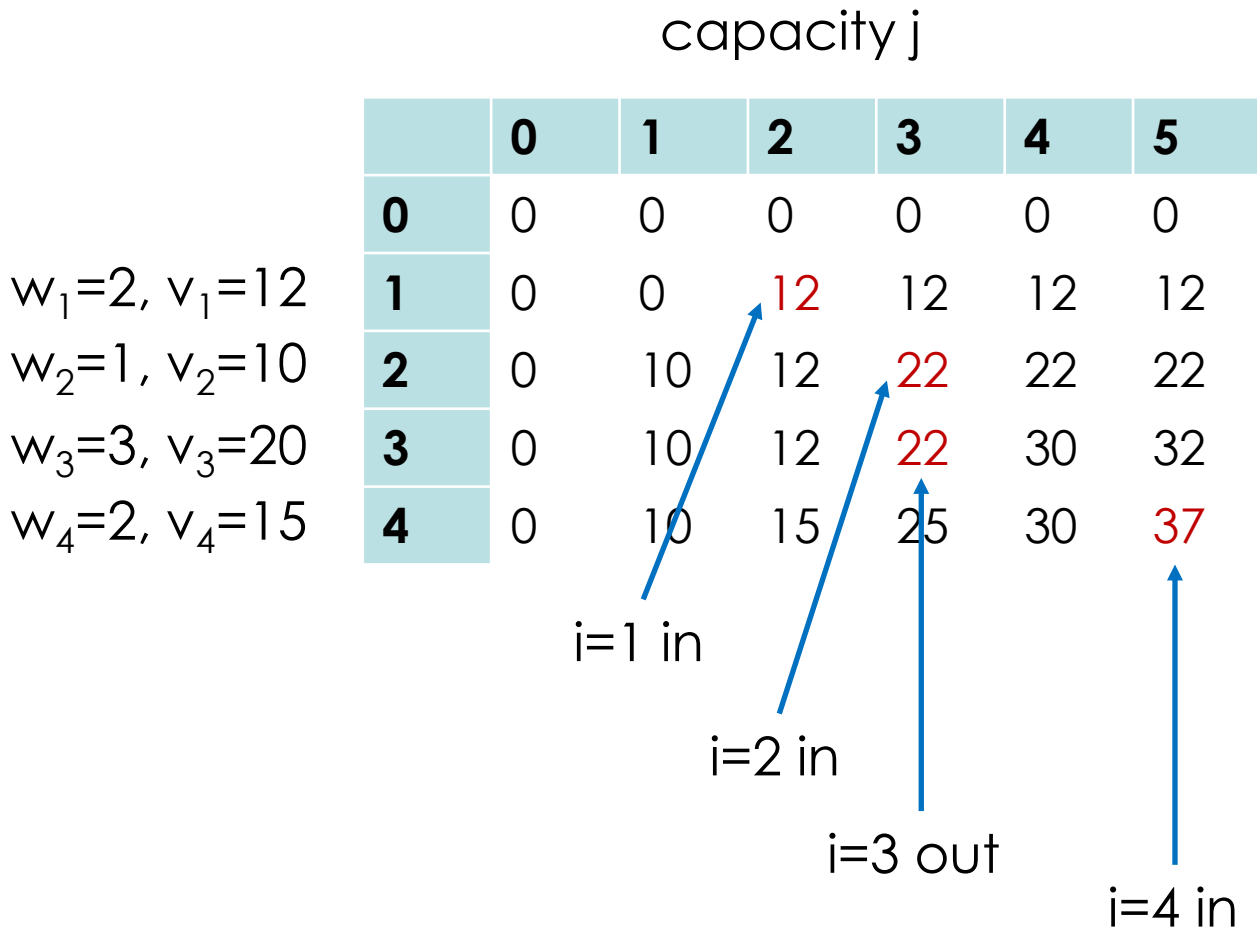
capacity j

Items i		0	1	2	3	4	5
	0	0	0	0	0	0	0
w_1=2, v_1=12	**1**	0	0	12	12	12	12
w_2=1, v_2=10	**2**	0	10	12	22	22	22
w_3=3, v_3=20	**3**	0	10	12	22	30	32
w_4=2, v_4=15	**4**	0	10	15	25	30	37

max(V[3,4],V[3,2]+15)
= max(30, 27) = 30

Knapsack Contents

- V[i,j] only provides knapsack values, not contents
- But we can work backwards
  - If $V[i-1,j] \neq V[i,j]$ then item i has been included, otherwise not
  - Then proceed accordingly to check $V[i-1, j-w_i]$ (i included), or $V[i-1,j]$ (i not included)



	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	37

Memory Function

- Dynamic programming solves problems with a recurrence relation that provides overlapping subproblems
- Top down (brute force)
  - Solves common subproblem more than once, leading to exponential calculations
- Bottom up (dynamic programming)
  - Fills table with all subproblems but calculates each one only once
- Memory functions combine both
  - Fill table with nulls, proceed top down and only calculate when needed
  - Equivalent to lazy evaluation

Knapsack Memory Function Solution

```
sack(i,j)
    if V[i,j] < 0 // -1 is null
        if j < w[i] then val ← sack(i-1,j)
        else val ← max( sack(i-1,j),
                        v[i]+sack(i-1,j-w[i]))
        V[i,j] ← val
    return V[i,j]
```

w_1=2, v_1=12
w_2=1, v_2=10
w_3=3, v_3=20
w_4=2, v_4=15

	0	1	2	3	4	5
0	0	**0**	**0**	**0**	**0**	**0**
1	0	**-1**	**-1**	**-1**	**-1**	**-1**
2	**0**	-1	**-1**	**-1**	-1	**-1**
3	0	-1	-1	**-1**	-1	**-1**
4	0	-1	-1	-1	-1	**-1**

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	**0**	**12**	**12**	**12**	**12**
2	0		**12**	**22**		**22**
3	0			**22**		**32**
4	0					**37**

Summary

- *Dynamic Programming* is typically applied to a recurrence relation
  - Break a problem down into smaller, more easily solved subproblems. To avoid solving these subproblems several times, their results are computed and recorded in a table.
  - Can only be applied when the principle of optimality holds
- the biggest limitation of Dynamic Programming is the number of partial solutions we must keep track of
  - The memory function tries to combine the strengths of top-down and bottom-up by doing only those subproblems that are needed and only doing them once
- Dynamic programming can be hard to formulate – often (simpler) greedy approaches do better